Building with AI

A one-page guide for anyone using AI builders like Replit, Lovable, or Base44.

Most people jump straight to *"build me an app."* The AI then guesses everything (who it's for, what it looks like, what it shouldn't do) and the guesses are usually wrong. **These four rules fix that.**

Rule 1. Set the context BEFORE asking for features

Your **very first message** in a new Replit project should not be *"build me..."*. It should be a full spec like the one below, **with photos of your storyboard and POC screens attached.**

Your storyboard and POC are the thinking you do *before* opening Replit, rough sketches of each screen and how they connect, on paper or in Figma. Photos of them carry far more intent than any paragraph of description. Have them ready before you paste.

The yellow boxes are what you swap with your own app's info. Everything else stays as-is. Paste the whole block as your first message, attach your storyboard and POC photos, then wait for the AI to acknowledge before you ask for any features.

FIRST MESSAGE (PASTE INTO REPLIT)

```
Build a web/native app called TuneShare, a peer-to-peer marketplace that
connects musical instrument owners with musicians who want to rent instruments.

I've attached photos of my storyboard and POC screens. Use them as the reference
for how each screen should look and how the user moves between them. The text below
covers features, monetization, branding, security, and what NOT to build.

Before you build anything, create an ARCHITECTURE.md file that explains in plain
language what the frontend is, what the backend is, what the database stores, and
how they connect. Keep it updated as the app grows.

Core features:
- Google login for sign-up
- Two user roles: Owners (list instruments) and Renters (browse and rent). A user
can be both.
- Listings: Owners post instruments with photos, description, category (guitar,
piano, drums, brass, strings), condition, hourly/weekly price, and pickup city
- Location matching: show renters the closest instruments first
- Booking & calendar: renters pick rental dates; show availability
- Handover flow: both parties confirm pickup and return
```

- **Payments via Stripe, with a refundable deposit held until return**
- **In-app chat between owners and renters to coordinate handover**
- **Reviews and ratings for both sides after a completed rental**

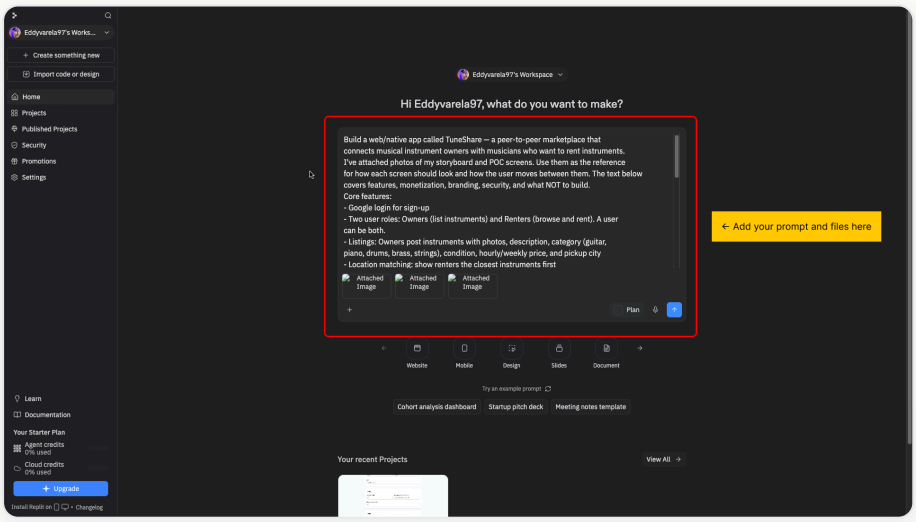
Monetization:
- **Revenue model: 10% platform commission on every completed rental, deducted before payout to the owner.**
- **Refundable security deposit (amount set per listing) held by the platform until the return is confirmed.**
- **No subscription tiers in v1, pay-per-rental only.** (If your app has plans, list each tier here: name, price, what's included.)

Branding & design:
- App name: **TuneShare**
- UI: keep it generic and functional with Tailwind UI. Use the AI builder's default look, no custom logos, colors, or fonts. (Branding will come later, in a separate prompt.)

Security: build this like a real app, keep secret keys (Stripe, DB, login) in environment variables on the server, never in browser code and never hard-coded in files; verify all user input and permissions on the server, not the client; stop and ask me if a step feels risky.

Don't add: **extra logins beyond Google, ads, long sign-up forms, paid tiers, dark mode toggle**

If anything is unclear, ask me before building.



This is what it looks like in Replit: paste the prompt in the message box, then attach your storyboard and POC screens.

Wait for the AI to acknowledge, then let it build.

> **Why this works:** you only say it once. Every feature you ask for afterwards inherits this

context. Without it, every prompt is a fresh coin flip.

> **Chat hygiene:** never paste real API keys, passwords, or anyone's personal data into the chat. Secrets belong in environment variables (Replit's Secrets tab) — the prompt above already tells the AI to keep them there.

Rule 2. One feature per task — and read the replies

After the first build, ask for **one change at a time, not a list**. And when you move on to a new feature, **start a new task** instead of continuing the old conversation.

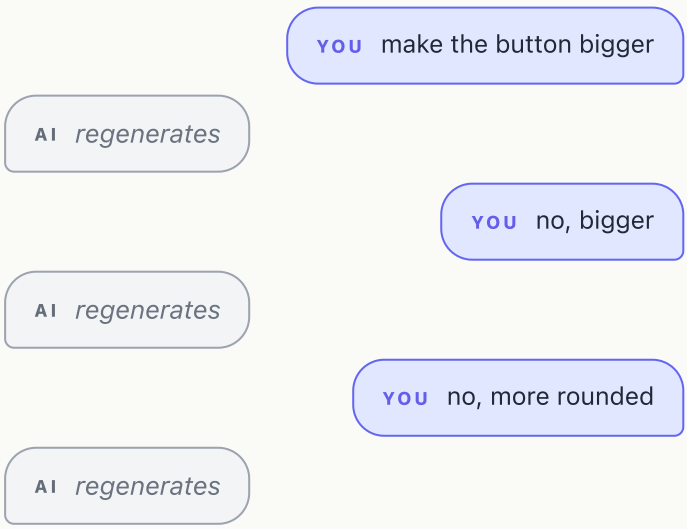
Long conversations fill up with dead ends and old mistakes — call it *context rot*. The AI re-reads all of it on every message, so it gets more confused *and* more expensive as you go. A fresh task loses nothing important: your spec from Rule 1 lives in the project's files and memory, not in the chat history.

- **One feature per message. One feature per task.**
- **Read every reply before sending the next message.** The AI tells you what it changed and often asks a question — skipping that is how bugs slip in.
- **Budget for it.** Building a real app costs real money: plan on the $100/month tier and keep an eye on your usage. Fresh tasks aren't just cleaner, they're cheaper.

Rule 3. When it's wrong, fix the rules, not the output

It's tempting to keep arguing with the AI about the same screen. That's a trap. You'll have the same fight on the next screen, and the next.

✕ THE TRAP



...next screen the AI makes, the buttons are small and square again.

✓ THE FIX (ADD A RULE INSTEAD OF ARGUING)

YOU Add this to your context and apply it from now on: **all buttons must be at least 48px tall and have rounded corners.** Then redo this screen.

AI ✅ Got it.

Every future screen has tall, rounded buttons. For free.

A few rules people commonly end up needing:

- "Never use red, it looks like an error."
- "Always show prices with the currency symbol."
- "When something succeeds, say it in plain language, not 'Success.'"
- "Error messages must say what went wrong and what to do next — never just 'Something went wrong.'"
- "Forms have at most 3 fields per screen."

> The output is the symptom. The rules are the cause. **Fix the cause.**

Rule 4. Keep a tiny "lessons" note

Open a Google Doc called **Lessons**. Every time you find a rule you wish you'd had on day one, write one line.

Examples:

- "I have to tell the AI who the user is, every time."
- "I ask for the mobile view first; it's the harder constraint."
- "'Make it cool' gives me random results. Two specific words work better."
- "If I don't say what NOT to add, the AI adds it by default."

Next project, paste your lessons into Rule 1 from day one. Your context gets smarter, the AI gets sharper, and you stop repeating the same fights with yourself.

THE WHOLE LOOP

Set the context → one feature per fresh task → if it's wrong, fix the rules not the output → write down what you learned

THAT'S IT.